

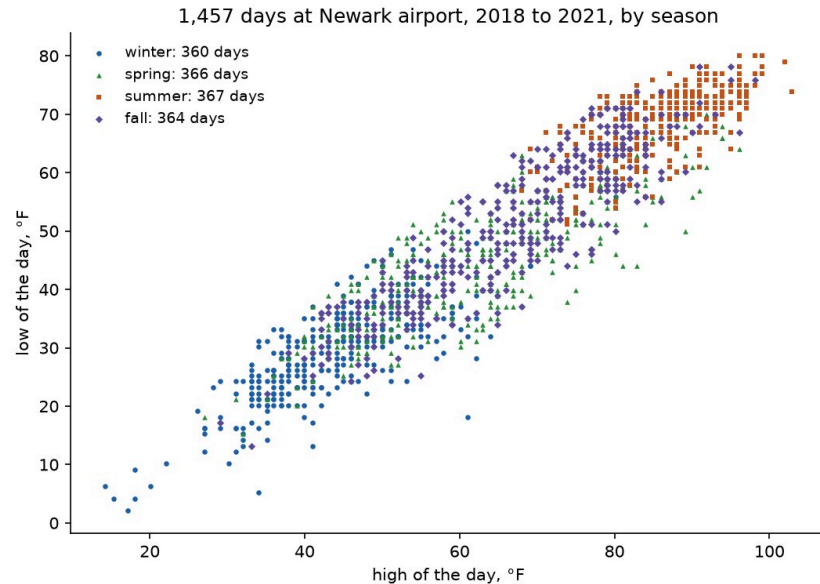
A Layer of Neurons Is a Matrix

Four neurons read a day of weather and name its season; five read a pixel and name its colour. Each layer is one matrix, and applying it to the data is one product.

The data: days at Newark airport

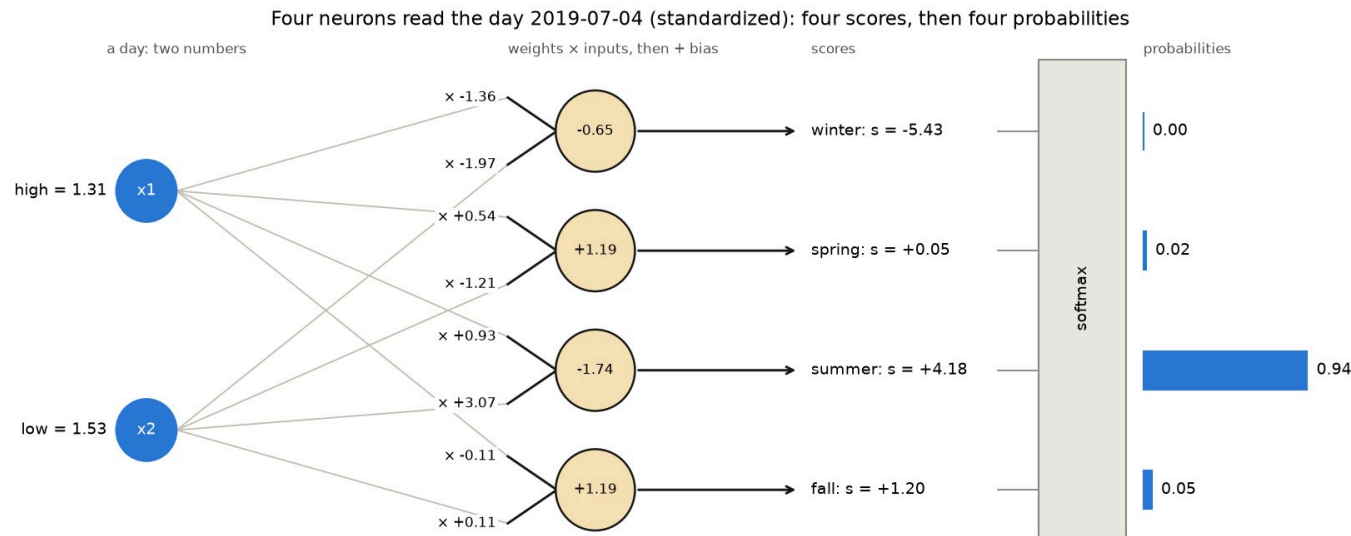
The days of 2018 to 2021 are $n = 1,457$ points with $m = 2$ coordinates each. The first four:

date	tmax_f	tmin_f
2018-01-01	20.1	6.3
2018-01-02	27.1	12.2
2018-01-03	30.2	10.2



- The daily weather at Newark airport, 2006 to 2024. One **point** is one day, with $m = 2$ coordinates, its high and its low; the $n = 1,457$ days of 2018 to 2021 are the rows of one matrix, standardized as in meeting 10 with the means and the deviations of 2006 to 2017.
- Each day carries a **label**, its season. Can four rules, one for each season, read the two numbers of a day and name its season?

A neuron: a rule that reads a day



- A **neuron** has one **weight** for each coordinate of the point and one more number, its **bias**. It multiplies each coordinate by its weight, adds the products and the bias, and the sum is its **score**. Four neurons, one for each season, read the same two numbers; the softmax after them turns the four scores into four probabilities.
- Nobody chose these weights by hand: a program adjusted them in many small steps until the layer named the right season for as many days of 2006 to 2017 as it could. That is **training**.

One neuron, one day: the score by hand

$$s = w_1x_1 + w_2x_2 + b$$

Day 2019-07-04: high 89 °F, low 74 °F. Standardized: $z = (1.31, 1.53)$.

The winter neuron: $w = (-1.36, -1.97)$, $b = -0.65$.

Its score: $-1.36 \times 1.31 + (-1.97) \times 1.53 + (-0.65) = -5.43$

shape of w : (2,), shape of z : (2,); $w @ z + b$ is one number.

- The winter neuron on the Fourth of July: a large negative score. The neuron **fires** when $s \geq 0$; this one does not.
- With matrices, w and x are **columns**, $m \times 1$, and $s = w^T x + b$: the **transpose** writes w as a row, and a row times a column is one number, the sum of the products of matching entries. In NumPy both have one dimension, shape (2,), and `w @ x + b` is their dot product plus the bias.

Four neurons, one day: the weights are a matrix

$$s = Wx + b$$

`W @ z + b, shape (4,): winter -5.43, spring +0.05, summer +4.18, fall +1.20`

`shape of W: (4, 2), shape of b: (4,)`

The largest score is that of the summer neuron, at position 2: the `argmax`.

- **Row j of W holds the weights of neuron j .** W is $K \times m = 4 \times 2$ and x is 2×1 , so Wx is 4×1 , and so is b : four scores in one line, `W @ x + b`. Nothing has to be repeated.
- The answer of the layer is the neuron with the largest score: the position of the largest entry, its **`argmax`**.

From scores to probabilities: the softmax

$$p_k = \frac{e^{s_k}}{e^{s_1} + e^{s_2} + \dots + e^{s_K}}$$

```
def softmax_rows(S):  
    shifted = S - S.max(axis=1, keepdims=True)    # the largest entry of every row becomes 0  
    E = np.exp(shifted)  
    return E / E.sum(axis=1, keepdims=True)
```

the four scores:	winter -5.43, spring +0.05, summer +4.18, fall +1.20	
their probabilities:	winter 0.000, spring 0.015, summer 0.937, fall 0.048	(they add to 1.000)
sigmoid of each score:	winter 0.004, spring 0.513, summer 0.985, fall 0.769	(each entry by itself)

- The K scores of a point become K positive numbers that add to 1, read as probabilities. Every probability depends on **all the scores of the point**: the softmax is a function of a row. On a matrix of scores it treats each row separately.
- The **sigmoid**, $\sigma(s) = 1 / (1 + e^{-s})$, is the other function of this unit. It acts on **every entry by itself**, replaces fires-or-not by a number between 0 and 1, and keeps the shape of its input. Its four numbers do not add to 1.

Four neurons, many days: one product

```
S_days = Z_days @ W_seasons.T + b_seasons      # the line of Step 4
```

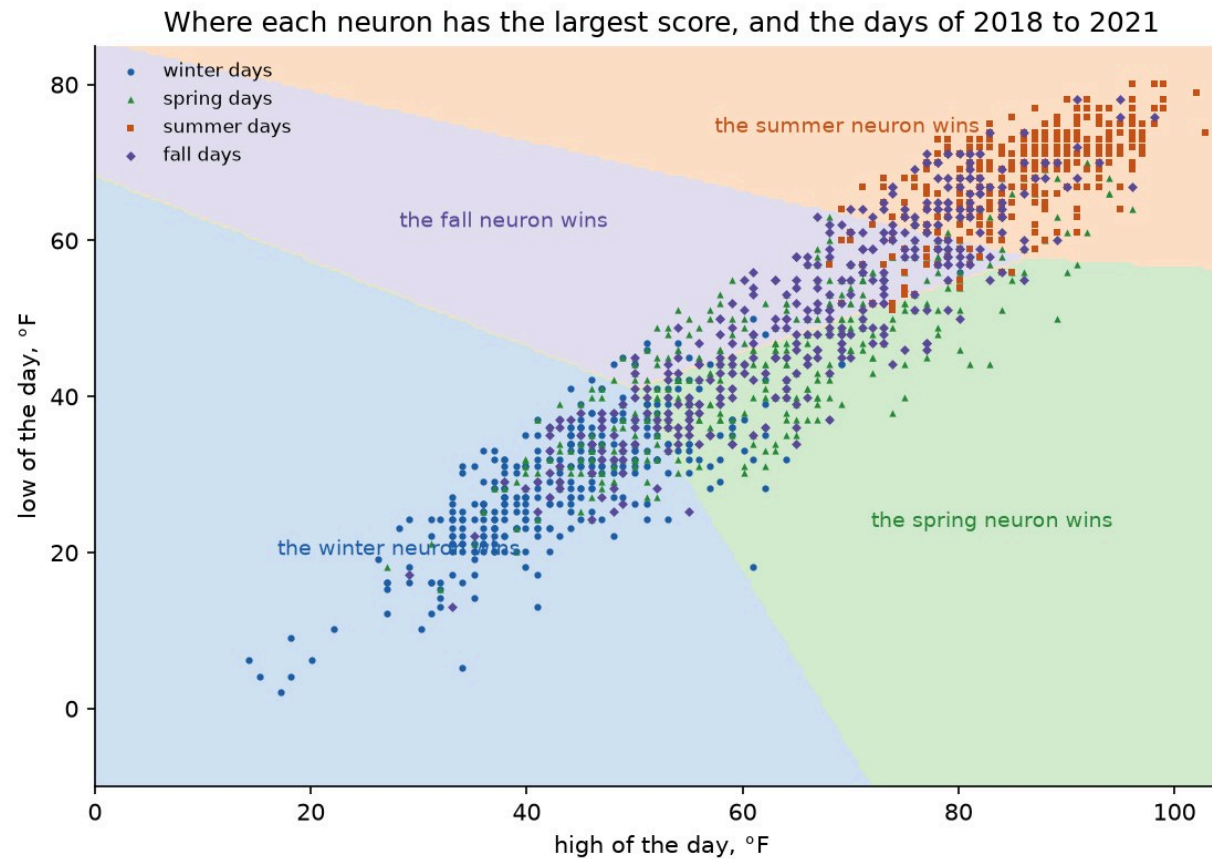
$$S = XW^T + \mathbf{1} b^T$$

shape of Z_days: (1457, 2) shape of W_seasons.T: (2, 4) shape of S_days: (1457, 4)

The largest score names the right season on 59.3% of the 1,457 days.

- X is $n \times m$ and W^T is $m \times K$, so S is $n \times K$: **row i belongs to day i, column j to neuron j**. W is transposed because a neuron is a row of W, and the product needs each neuron's weights as a column.
- $\mathbf{1}b^T$ is the row of biases written again on every row: the mathematics needs the column of ones, and NumPy **broadcasts** `+ b`, as it broadcast `- mu` in Act 0.

Where each neuron wins



The plane of highs and lows, coloured by the neuron with the largest score. The days are drawn in the colour and the marker of their true season; the layer gets 59.3% of them right.

The four cases

	one point x , shape $(m,)$	n points X , shape (n, m)
one neuron: w of shape $(m,)$, b a number	$w @ x + b$ is a number	$X @ w + b$ has shape $(n,)$; b is broadcast
K neurons: W of shape (K, m) , b of shape $(K,)$	$W @ x + b$ has shape $(K,)$	$X @ W.T + b$ has shape (n, K) ; b is broadcast

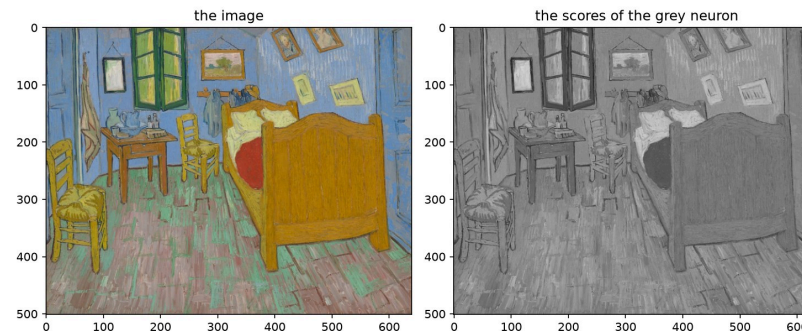
- The bias needs help exactly when there are many points: the mathematics writes a column of ones, and NumPy broadcasts.
- The shapes do not always warn of a missing transpose: when $K = m$, $X @ W + b$ runs, and its numbers are wrong.
- The same four lines work on any data. The second example: the pixels of your group's image.

A second example: a neuron that scans an image

```
X = image.reshape(-1, 3)          # one row for each pixel
n, m = X.shape
print(f"shape of X: {X.shape}, which is n = {n:,} points with m = {m} coordinates")

s_grey = X @ w_grey + b_grey       # the scores of the grey neuron on all the points
print("shape of X @ w_grey + b_grey:", s_grey.shape)
```

shape of X: (320640, 3), which is n = 320,640 points with m = 3 coordinates
shape of X @ w_grey + b_grey: (320640,)

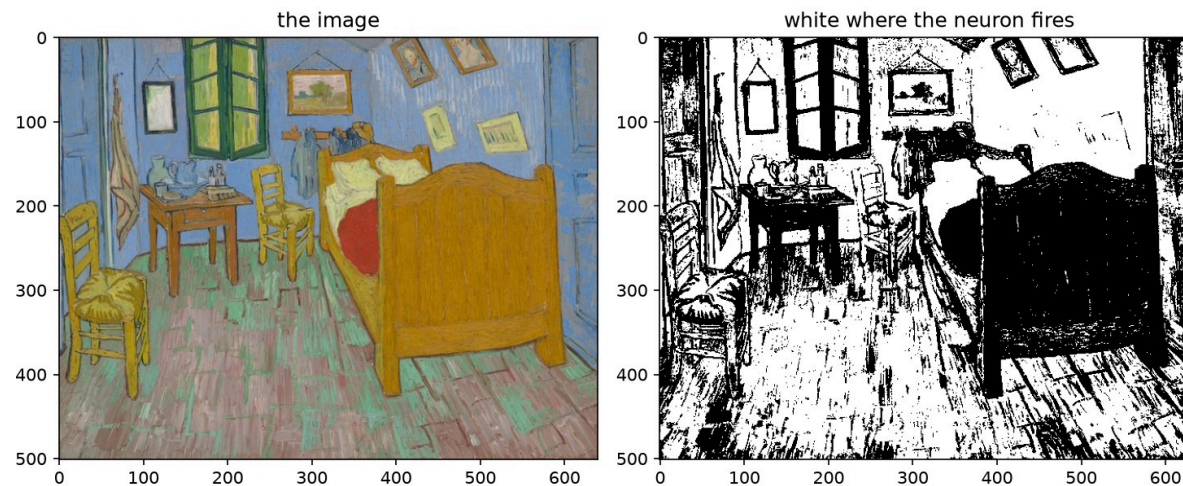


A point is now one **pixel**, with $m = 3$ coordinates: red, green, blue. `image.reshape(-1, 3)` writes the $n = H \times W$ pixels as the rows of X . The neuron with the weights 0.299, 0.587, 0.114 and the bias 0 reads every pixel, and its score is the brightness: the picture of the scores is the grey image of meeting 10.

A neuron that fires

```
b_bright = -s_grey.mean() # minus the mean brightness of the image
fires = X @ w_grey + b_bright >= 0 # True for the points on which the neuron fires
print(f"bias {b_bright:.3f}: the neuron fires on {fires.mean():.1%} of the {n:,} pixels")
```

bias -0.518: the neuron fires on 53.1% of the 320,640 pixels



A brightness is never negative, so with the bias 0 the grey neuron fires on every pixel. With the bias $-(\text{mean brightness})$ it fires on the pixels brighter than the average. $\theta = -b$ is the **threshold** of the neuron.

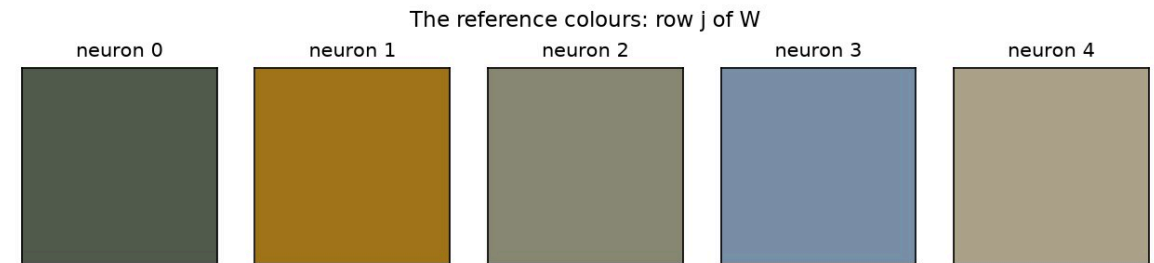
Five neurons, one for each reference colour

```
K = 5
C = reference_colours(X, K)
W = C.copy()
b = -0.5 * (C ** 2).sum(axis=1)
```

K rows and 3 columns: the reference colours, darkest first
the weights of the layer: row j is reference colour j
the biases of the layer: one number for each neuron

shape of W: (5, 3) shape of b: (5,)

```
W =
[[0.32 0.36 0.31]
 [0.63 0.46 0.1 ]
 [0.53 0.53 0.45]
 [0.48 0.57 0.66]
 [0.67 0.65 0.55]]
b = [-0.16 -0.31 -0.38 -0.49 -0.59]
```



Five **reference colours** of the image, found by a method called k-means. Neuron j has the weights c_j and the bias $-\frac{1}{2}|c_j|^2$: **the largest score belongs to the neuron whose colour is nearest to the pixel**. The layer names the colour of a pixel, as the layer of seasons named the season of a day.

The scores of one pixel

$$s_x = W @ x + b$$

the K scores of the pixel x

the pixel $x = [0.56 \ 0.39 \ 0.1 \]$

shape of $W @ x + b$: (5,)

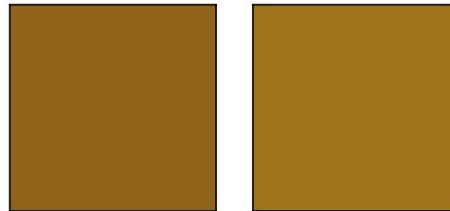
the scores of the 5 neurons: $[0.19 \ 0.237 \ 0.17 \ 0.065 \ 0.104]$

The largest score is that of neuron 1, whose colour is $[0.63 \ 0.46 \ 0.1 \]$.

The pixel, and the colour with the largest score

the pixel x

colour of neuron 1



$W @ x + b$, as for the four neurons and one day. Positions are counted from 0, so the neurons are numbered 0 to 4.

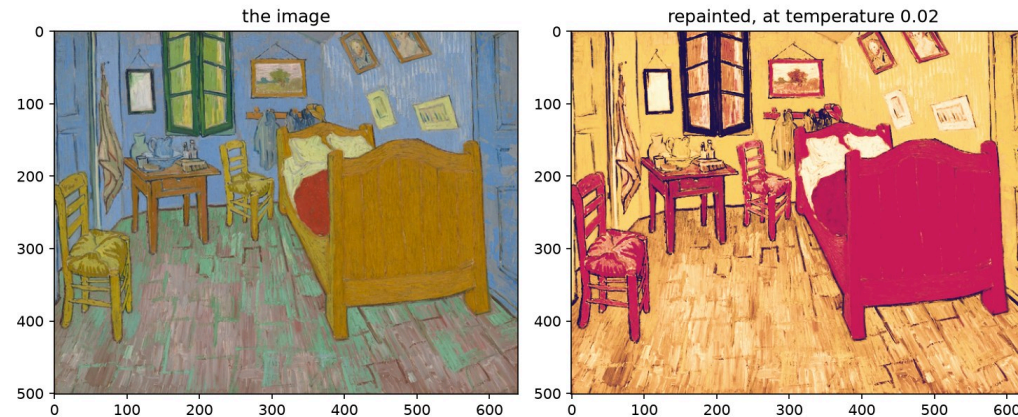
Many neurons, many points: the three loops

```
def layer_loops(X, W, b):  
    n, m = X.shape  
    K = W.shape[0]  
    S = np.empty((n, K))  
    for i in range(n):                # every point  
        for j in range(K):            # every neuron  
            total = b[j]  
            for c in range(m):         # every coordinate  
                total += W[j, c] * X[i, c]  
            S[i, j] = total  
    return S
```

- S has n rows and K columns: **row i belongs to pixel i, column j to neuron j**. The loops compute it from the definition: every point, every neuron, every coordinate.
- For the days, the same scores were one line: $S = XW^T + \mathbf{1}b^T$. **In Colab you write that line for the pixels**, and the check compares it with the loops.

Repaint the image: a small network

```
Prob = softmax_rows(layer(X, W, b) / T)    # n rows and K columns  
new = Prob @ PAINT                        # n rows and 3 columns
```



- Each neuron gets a **paint**, a colour of the group's choice: the rows of `PAINT`, $K \times 3$.
`Prob @ PAINT` is $n \times 3$: the new colour of a pixel is a mixture of the K paints, each counted with the probability of its neuron.
- The second product is of the same kind as the first: n points, now with K coordinates, and three neurons. **Two layers with a function between them are a small neural network**, applied to all the pixels at once.

Now in Colab: Act 1

- The class notebook, with the GPU asked for, the setup run and your group's number typed, as at the start.
- **Act 1:** run the cell that builds the layer of five neurons on your image. Then **write the one line** of the function `layer`: the three loops as one product, as for the days. The check tells you which array to transpose.
- Repaint your image: choose five paints, and try the temperatures 1, 0.1 and 0.002.
- **Post** on the discussion board: the printed line, the picture of your image beside the repainted one with one sentence on what it shows, and one sentence on what the temperature changed. One member posts; begin with your group's number and the names of all its members.

Loops against the product

`n = 320,640 points, m = 3 coordinates, K = 5 neurons`

`The innermost line of the loops runs  $n \times K \times m = 4,809,600$  times.`

<code>version</code>	<code>seconds</code>
----------------------	----------------------

<code>three loops</code>	<code>2.5945</code>
--------------------------	---------------------

<code>one product</code>	<code>0.0093</code>
--------------------------	---------------------

`The loops take 278 times as long as the product.`

- The innermost line of the loops runs $n \times K \times m$ times; the product carries out the same multiplications and additions, inside NumPy. Meeting 12 counts them, and measures what a second of computing is worth on each processor.
- Act 2: the same line in PyTorch, and on a GPU.